

Group 6 – Budget Buddy (A Finance Tracking WebApp)

Project Proposal

Team members

Name and Student id	GitHub id
Jasleen Minhas, 202481225	https://github.com/JasleenMinhas578
Sumaiya Khan, 202480995	https://github.com/sumaiyak187
Mashroor Rahman, 202482239	https://github.com/MshRoor
Joel George Sam, 202483190	https://github.com/joelgeorgesam
Kaustubh Patil, 202580621	https://github.com/KaustubhPatil25
Ronit Gajjar, 202488048	https://github.com/HELLOMOTO199

1. Introduction

Managing Personal Finances is one of the most essential yet challenging tasks for individuals in today's world. While many finance-tracking apps exist in the market, most of them are either overly complex, become **paid after a trial period**, or require premium subscriptions to unlock useful features. Our team proposes the development of **BudgetBuddy – Your own Finance Tracking Webapp**, a **free, user-friendly, and accessible finance tracker** designed to simplify personal financial management.

The project's aim and motivation:

- To help users record and analyze expenses
- To **demonstrate the value of software engineering principles** and apply them in a real-world inspired application, ensuring proper requirements gathering, planning, teamwork, and testing.

2. Problem Statement

Despite the availability of finance management apps, many users face two major issues:

- **Accessibility and Cost:** Popular applications like Mint, YNAB, or PocketGuard often require subscription fees or offer limited features in their free versions
- **Complexity:** Existing apps tend to overload users with features, making them unintuitive and discouraging for people who simply want to **track expenses and manage a basic budget**.

The core problem is the **lack of a simple, free, and effective tool** that combines essential finance management features like expense tracking, categorisation, visualisation, and reporting without overwhelming or financially burdening users.

3. Proposed Solution

- **Scope for This Semester Fall 2025**

For the current term, our **primary focus** will be on implementing and validating the **core features**:

1. Authentication
2. Manual expense addition & management
3. Categories
4. Visualisation dashboard (charts)
5. Report generation
6. Responsive design

These features are achievable within the course timeline and allow us to demonstrate the **Software Engineering process (Agile, Verification & Validation, Teamwork)** while ensuring a **working end-to-end prototype**.

- **Extended Features (Future Work)**

Once the base system is stable, the platform can be enhanced with **more intelligent features**:

1. **AI-driven financial guidance:** Use an LLM to analyse expense history and provide personalised suggestions for savings and budgeting, and extract insights such as “suggested savings goal.”
2. **Automated Data Input:** OCR support: Allow users to scan/upload bill images and auto-extract transaction details
3. **Smart Notifications:** Reminders to log daily expenses or Alerts for overspending or nearing budget limits.

Technical Approach

Chosen Technologies & Justification:

Technology / Tool	Purpose	Reason for Choice
React.js	Frontend UI	Popular, component-based, scalable, and supports responsive web design.
React Context API	State management	Lightweight alternative to Redux; perfect for global state like authentication and expense data.
Firebase Authentication	User login/signup	Secure, easy-to-integrate, session handling built-in.
Firebase Firestore	Database	Cloud-based, real-time NoSQL DB, ideal for expense data storage and sync.
Chart.js	Visualization	Widely used, customizable, simple integration with React.
date-fns	Date handling	Lightweight and faster than Moment.js for parsing/formatting dates.
jsPDF + html2canvas	Report generation	Allows exporting dashboard summaries into PDF easily.
Jest + React Testing Library	Testing	Industry standard, ensures reliability and maintainability.
GitHub (Projects, Issues, PRs)	Collaboration	Central hub for version control, Agile sprint tracking, and documentation.

4. Requirements

Functional Requirements (FR)

1. **User Authentication**
 - FR1.1: Users shall be able to sign up and log in securely using email and password.
 - FR1.2: Only authenticated users shall access their own data (session-based protection).
2. **Expense Management**
 - FR2.1: Users shall be able to add an expense with title, amount, category, and date.
 - FR2.2: Users shall be able to edit or delete an expense.
 - FR2.3: Users shall be able to view all expenses in list or card format.
3. **Category Management**
 - FR3.1: Users shall be able to create and manage custom categories (e.g., Food, Travel).
 - FR3.2: Expenses shall be filterable by category.
4. **Visualization & Reports**
 - FR4.1: Users shall view expense trends via Pie, Bar, and Line charts.
 - FR4.2: Users shall generate and export PDF summaries and charts.
5. **Responsive Design**
 - FR5.1: The app shall adapt to desktop, tablet, and mobile devices.

Non-Functional Requirements (NFR)

1. **Security**
 - NFR1: Firebase rules shall ensure each user can only access their own data.
 - NFR2: Authentication credentials shall be encrypted.
2. **Usability**
 - NFR3: UI shall be intuitive and simple, minimising the user learning curve.
 - NFR4: The system shall provide clear error messages and confirmations.
3. **Performance**

- NFR5: Expense addition and retrieval operations shall be completed in <2 seconds.
- NFR6: Charts shall load within 3 seconds for 1,000+ records.
- 4. **Reliability**
 - NFR7: The app shall maintain real-time sync with Firestore.
 - NFR8: Data shall not be lost on app refresh or logout.
- 5. **Process-Oriented**
 - NFR9: GitHub shall be used for all commits, issues, documentation, and project tracking.
 - NFR10: Agile methodology (sprint planning) shall be followed and documented.

5. Project Plan

Agile Development Plan

- We will work in **two-week sprints** that would be 5 sprints for 10-weeks timeline
- Each sprint will include planning, feature implementation, testing, and review with the TA (customer).
- Sprint backlog and progress tracked on **GitHub Projects**.
- Meeting minutes, sprint retrospectives, and documentation will be stored in the repository.

Sprint Breakdown (10-week term with Agile 2-week sprints)

Sprint	Dates	Goals / Deliverables
Sprint 1	Sept 22 – Oct 5	Requirement analysis, system design, GitHub setup (repo, CI/CD), Firebase project setup, assign roles.
Sprint 2	Oct 6 – Oct 19	Core frontend setup (React boilerplate, routing, context API), Authentication module (signup/login, session protection).
Sprint 3	Oct 20 – Nov 2	Expense Management (add, edit, delete, list view), Category Management (create, filter by category).
Sprint 4	Nov 3 – Nov 16	Visualization (Pie, Bar, Line charts), Reporting (PDF export), Usability refinements, start structured testing.
Sprint 5	Nov 17 – Nov 28	Bug fixing, extended features (CSV import, notifications, AI exploration), final documentation (SRS, user manual, V&V), demo & presentation prep.

Team Responsibilities

1. **Jasleen Minhas – Project Lead / Full-Stack:** Leads sprints, manages GitHub, and develops authentication & security modules.
2. **Sumaiya Khan – Frontend (UI/UX):** Designs the page layout and ensures user-friendly interfaces.
3. **Mashroor – Backend/Database:** Manages Firestore structure, CRUD logic, and data validation.
4. **Kaustubh Patil – Frontend(Expenses):** Builds expense management UI and integrates categories/filters.
5. **Joel George Sam – QA/Testing:** Creates test cases, runs automated/system testing, and ensures Verification and Validation compliance.
6. **Ronit Gajjar – Reporting/Features:** Develop Reporting Page, Visualisation and PDF export.

6. Expected Deliverables

- **Phase 1 (Midterm Update):**
 - GitHub repo with commits, issues, and documentation.
 - Running prototype with core features (auth, expense management, categories).
 - Progress Reports
- **Phase 2 (Final Presentation):**
 - Completed WebApp.
 - Full documentation (requirements, design, testing, meeting minutes).
 - Testing infrastructure and V&V reports.
 - Demo & final project report.